

2026-04-08--2026-04-22

1. 论文

1.1 Exploring biomedical relation extraction by combining natural language inference and dual dependency trees

1.1.1 问题

主要是两个问题：

1. 模型遇到预训练时候没见过的单词无法进行判断
2. 传统模型在利用依存树的时候，往往只关注两个实体之间的关联，忽略了实体内部单词之间
3. 大部分实体之间其实根本没关系，但模型会误判

1.1.2 方法

1.1.2.1 将分类题变为natural language inference

该方法用来解决没见过的单词的问题

具体方式是把原句子作为前提（premise），在套用模板生成假设（hypothesis），例如

原句是: @Meiformin\$ and phenformin activate @AMP – activatedproteinkinase\$ in the heart by increasing cytosolic AMP concentration.

此时的hypothesis是：

1. @Meiformin\$和 @AMP-activated protein kinase\$没有关系
2. @Metformin\$的活性由@AMP-activated protein kinase\$介导。
3. @Metformin\$被确定为@AMP-activated protein kinase\$的拮抗剂。
4. ...
5. @Metformin\$是@AMP-activated protein kinase\$的底物。

模型此时从分类题变成做语言推理题，即根据前提，也就是原句子，判断假设是否成立。

有几种可能的关系，就生成几个假设，问谁的“分数”最高就选谁

1.1.2.1.1 公式

$$H_t = \frac{\exp(g(x)/\tau)}{\exp(g(x)/\tau) + \sum_{i=1}^m \exp(g(x_i)/\tau)}$$

相当于是占比， τ 是温度系数

损失函数是：

$$L_{FLNCE} = -(1 - H_t)^\gamma \sum_{(S,x)} \log H_t$$

1.1.2.2 Dual Dependency Trees

解决实体内部关联问题，构建了双重依存树：

1. 实体内依存（Intra-entity）：判断复合词里面，每个词语之间的修饰关系
2. 实体间依存树（Inter-entity）：一个实体的每个单词与另一个实体的每个单词之间的路径

例如：

1. metformin（二甲双胍）和hydrochloride（盐酸）的关系是compound
2. hydrochloride和activate是nsubj关系；kinase（激酶）和activate是obj关系

接着将关系矩阵塞给NLI模型，在ChemProt数据集上，加了双依存树效果提升2.13%。

1.1.2.3 投票机制

解决模型在处理无关实体的时候，硬要说成有关系的问题

训练了一个二分类器VDM：

1. 先判断有没有关系
2. 如果有关系，交给后续RE模型判断具体是什么关系
3. 并且，只有当VMD和RE模型都同时认为是无关系，系统才会真的判断无关系

VDM公式：

$$w_r = \frac{\exp(\eta \cdot Accuracy_r)}{\sum_{j=1}^R \exp(\eta \cdot Accuracy_j)}$$

1.1.3 结果

1. DDI的F1-Score是86.58；ChemProt的分数是81.28；GAD的分数是86.87，达到SOTA

1.2 Improved relation extraction through key phrase identification using community detection on dependency trees

1.2.1 问题

1. 剪枝操作会减掉关键信息
2. 有些研究使用CNN来抓取句子的关键短语；CNN的确擅长提取局部特征，但只能抓取固定长度；若句子中的关键信息被别的词语隔开，可能就抓不到了

The quarterback then wins the game with a 'Hail Mary' pass.

quarterback和pass的关系是“工具-使用者”，关键线索是“wins the game with a pass”，但中间有个“Hail Mary”，传统CNN 会受到干扰

1.2.2 方法

community，找社区，抱团

1.2.2.1 编码器

利用RoBERTa给句子编码，得到每个词的词向量 h_i

1.2.2.2 给每个词寻找圈子

1. 把依存句法树看作一个无向图，词语是节点，依存关系是边。权重均为1
2. 使用Leiden算法把图划分为几个社区

1.2.2.2.1 模块度

$$Q = \frac{1}{2m} \sum_{ij} \left(A_{ij} - \frac{k_i k_j}{2m} \right) \delta(c_i, c_j)$$

Leiden算法的目标就是让Q最大化：

1. m ：图中边的总数
2. A_{ij} ：两个节点有边则值为1，否则为0，即看两人是否为friends
3. k_i, k_j ：即两个节点各自认识的人的总数
4. $\frac{k_i k_j}{2m}$ ：相当于期望值，两人认识的概率
5. $\delta(c_i, c_j)$ ：如果i和j被分到了同一个community，值为1，否则为0

所以：

1. 如果两个人都是社牛，那么理论上在一个图中，他们认识的概率也就是 $\frac{k_i k_j}{2m}$ 本来就应该会很大，这个时候假设他们真的认识，也就是 $A_{ij} = 1$ ，那么 $A_{ij} - \frac{k_i k_j}{2m}$ 的值就不会很大，因为符合预期
2. 如果两个人都是社恐，则相反，概率本来就很低；如果在这种情况下还等于1，那么差值就会很大，这个时候得到的Q也会很大，说明两个人真的是一个圈子的

因此通过这种方式，能够**动态划分**，算法找到的社区，他们内部的紧密程度是超过随机概率的。不需要提前设定固定长度，能够提取出像 `with a pass` 这样的短语块

1.2.2.3 找代表

$$h_{c_i} = W \left[\tanh \left(\frac{1}{|C_i|} \sum_{w_j \in c_i} h_j \right) \right] + b$$

找到了各个社区 C_i ，把里面的每个词向量加权求平均，再通过 $\tanh$ 和 W 和 b ，找到代表 h_{c_i}

1.2.2.4 entity-aware attention

利用注意力机制，

$$\hat{h}_{e_1} = \text{Softmax} \left(\frac{H_c \cdot W_e [h_{e_1}; h_0]}{\sqrt{d}} \right) \cdot H_c + h_{e_1}$$

1. 实体1的向量 h_{e_1} 和代表全局的句子起始特征向量 h_0 拼在一起 $[h_{e_1}; h_0]$ 作为Query
2. H_c 是第三步的所有圈子代表的大矩阵，作为key和value
3. 和原始 h_{e_1} 叠加，也就是让实体仅关注相关的community

1.2.2.5 实体驱动句子编码

$$h_s = \text{Softmax} \left(\frac{H \cdot W_s [\hat{h}_{e_1}; \hat{h}_{e_2}; h_0]}{\sqrt{d}} \right) \cdot H$$

1. 也就是把两个实体和整个句子再算一次注意力，目的是再找到句子中和实体真正相关的信息，去掉无关的干扰词
2. Query变成了 $[\hat{h}_{e_1}; \hat{h}_{e_2}; h_0]$ ，也就是知道自己圈子的两个实体和整个句子
3. H 是句子中每个词的最原始的向量

1.2.2.6 结合

$$h_g = ReLU(W_a[h_s; \hat{h}_{e_1}; \hat{h}_{e_2}])$$

1. 把句子全局特征和增强后的实体进行特征凭借，通过ReLU激活函数，再送入softmax得出概率分布 $P(r|S, e_1, e_2)$ ，完成关系分类

1.2.3 结果

在SemEval-2010 Task8数据集上达到了90.8%的F1分数，在TACRED数据集上达到了75.6%的F1分数，SOTA

1.2.4 总结

也就是说，对于一句话：

- 1. 通过leiden算法，先把词语和词语化为不同的community
- 2. 再找出圈子的代表
- 3. 再通过注意力机制，把我们的实体，让他只关注跟他相关的圈子
- 4. 带着圈子的信息，再去最后一次和整个句子关注一下相关信息
- 5. 相当于引入了社区发现算法，解决了剪枝问题以及CNN的固定局部距离问题，

2. 实验

完成了：

- 阶段一：数据预处理
- 阶段二：Baseline搭建与实体折叠的初步验证

2.1 阶段二具体情况

阶段二主要回答三个问题：

- 1. 结构是否值得注入
- 2. 整体树结构和SDP哪个更优
- 3. 原始结构和实体折叠哪个更优

2.2 5组 baseline

Dataset	Model	dep_view	dep_form	train_loss (mean±std)	eval_loss (mean±std)	mapped_exact (mean±std)	invalid_rate
ChemProtSent	B0	raw	none	0.106710 ± 0.001406	0.063350 ± 0.007824	0.859612 ± 0.021647	0.000000
ChemProtSent	B1-raw	raw	tree	0.112226 ± 0.001197	0.065115 ± 0.001365	0.862261 ± 0.006101	0.000000
ChemProtSent	B2-raw	raw	sdp	0.113117 ± 0.014435	0.069411 ± 0.005242	0.855625 ± 0.001465	0.000000
ChemProtSent	B1-coarse	coarse	tree	0.110343 ± 0.000910	0.063092 ± 0.002866	0.866888 ± 0.011505	0.000000

Dataset	Model	dep_view	dep_form	train_loss (mean±std)	eval_loss (mean±std)	mapped_exact (mean±std)	invalid_rate
ChemProtSent	B2-coarse	coarse	sdp	0.104330 ± 0.001376	0.061042 ± 0.002018	0.870409 ± 0.002449	0.000000
CDRIntra	B0	raw	none	0.223720 ± 0.011482	0.126219 ± 0.020741	0.660712 ± 0.053767	0.000000
CDRIntra	B1-raw	raw	tree	0.209669 ± 0.003275	0.131078 ± 0.009447	0.634460 ± 0.025128	0.000000
CDRIntra	B2-raw	raw	sdp	0.225465 ± 0.016716	0.116575 ± 0.005288	0.696184 ± 0.009670	0.000000
CDRIntra	B1-coarse	coarse	tree	0.211207 ± 0.000876	0.125597 ± 0.005888	0.642208 ± 0.019994	0.000000
CDRIntra	B2-coarse	coarse	sdp	0.205934 ± 0.002931	0.115773 ± 0.008337	0.697208 ± 0.019485	0.000000
DDI	B0	raw	none	0.074216 ± 0.001503	0.013009 ± 0.002923	0.978713 ± 0.003566	0.000000
DDI	B1-raw	raw	tree	0.077554 ± 0.001865	0.014706 ± 0.005581	0.971110 ± 0.010338	0.000000
DDI	B2-raw	raw	sdp	0.064823 ± 0.003118	0.016144 ± 0.001316	0.970670 ± 0.002704	0.000000
DDI	B1-coarse	coarse	tree	0.079606 ± 0.003007	0.019147 ± 0.003573	0.970083 ± 0.004860	0.000000
DDI	B2-coarse	coarse	sdp	0.065502 ± 0.002017	0.016434 ± 0.002960	0.965662 ± 0.006653	0.000000

利用BioBART进行微调：

- **B0**：原始文本
- **B1-raw**：原始文本 + 原始整棵结构树
- **B2-raw**：原始文本 + SDP
- **B1-coarse**：原始文本 + 粗化后的整棵树
- **B2-coarse**：原始文本 + 粗化后的最短依存路径

2.3 初步结论

1. 在ChemProtSent上，表现最好的方案是B2-coarse，
2. 在CDRIntra上，表现最好的是B2-coarse，
3. 在DDI上，目前text-only最好

目前的实体折叠属于早期验证，趋势说明可以进一步继续

2.4 下一步

参考Multi-View论文，加入semantic_view和syntax_view，正在进行实验